

Clash-A-Thon 2026

The Friction We Forget

Building Solutions for Problems We've Stopped Noticing

Kick-off
Feb 24, 2026 | 09:30 AM

Submission Deadline
Feb 26, 2026 | 4:00 PM

Final Presentation
Feb 27, 2026 | On-site

TEAM & PROJECT INFORMATION

Team Name	eXe
Project Title	OmniForm
Project Category	Web application

TEAM MEMBERS

#	Full Name	Role / Responsibility	GitHub / Contact
1	Sneha Agrawal	Business Roles	9804906264
2	Prashan Mainali	Logic / Storage layer development	prasan6969
3	Milan Timalsena	Interface layer development	milantimalsena
4	Sulav Giri	Interface layer development	sulavgirii02
5	Neek Kafle	Logic / Storage layer development	iamneek

SUBMISSION LINKS

GitHub Repository	https://github.com/Prasan6969/ClashAThon-eXe-OmniForm
Deployment URL	https://clash-a-thon-e-xe-omni-form.vercel.app/
Tech Stack	MERN Stack

Declaration

We declare that this project is our original work developed during Clash-A-Thon 2026. All external resources and libraries have been appropriately credited. We understand that plagiarism or misrepresentation will result in disqualification.

Table of Contents

1	Technical Documentation	3
1.1	Section A: System Architecture	3
1.2	Section B: Component Breakdown	4
1.2.1	Major Modules and Services.....	4
1.3	Section C: Data Design.....	5
1.3.1	Object Relation Data Structure Description	5
1.4	Section D: API/Interface Specifications	7
1.5	Section E: Setup and Deployment	8
1.5.1	Local Setup Commands.....	8
1.5.2	Deployment Process.....	9
2	Business Documentation	10
2.1	Section F: Business Model Canvas	10
2.2	Section G: Unique Selling Proposition	11
2.3	Section H: Market Analysis	11
2.4	Section I: Sustainability & Future Scope	13

Table of Figures

Figure 1: System architecture diagram.....	3
Figure 2: API Endpoints Description.....	7
Figure 3: BMC canvas.....	10

1 Technical Documentation

1.1 Section A: System Architecture

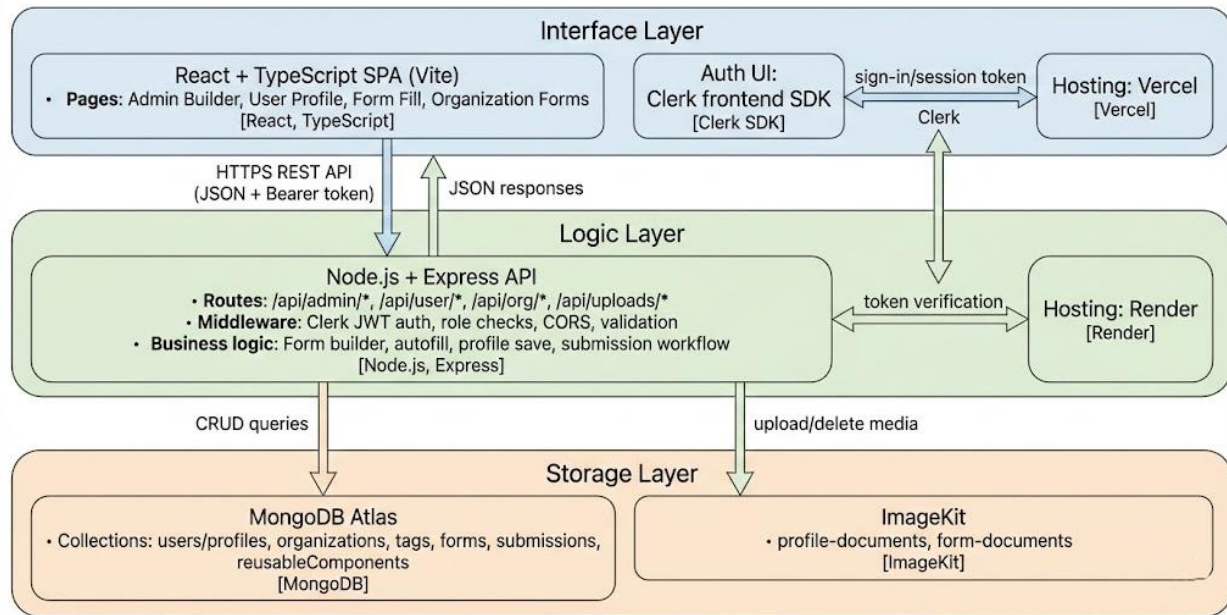


Figure 1: System architecture diagram

The diagram for system architecture presented above shows all three layers of the application: Interface layer, Logic layer, and Storage layer. The interface layer uses React.js and typescript using vite ecosystem. The logic layer uses Node.js with express. The storage layer uses MongoDB atlas for storage of non-media data, and for medias, ImageKit service is used. The arrows in the diagram above define the logic flow in detail.

1.2 Section B: Component Breakdown

1.2.1 Major Modules and Services

Frontend Web (React and Typescript, Vite)

- Renders UI for all the user types: Admin, Organization, and User.
- Routing is handled, features like profile editing, viewing and interacting with forms, and form builders (form building is for admin only).
- Interacts with backend APIs using properly authenticated request.

Backend Server (Node JS and Express)

- Allows access to its APIs by exposing it to the frontend.
- The backend server is responsible for enforcing authorization, validating requests, etc.
- Data coordination between the webapp and the storage layer.

Authentication Service (Clerk)

- Provides easy to maintain and developer friendly environment for managing authentication related services to the web app.
- Provides user identity and role related context, that is later used by the backend while processing request.

Storage Service (MongoDB and ImageKit)

- ImageKit was used to store media like pictorial documents and user profile images.
- Organizations, forms, tags, submissions, user profiles, etc. are stored in MongoDB.

Communication Between Components

- Frontend → Backend (*communication using the REST API*)
- Backend ↔ Clerk (*Verification of roles before processing requests*)
- Backend ↔ MongoDB (*For CRUD operation*)
- Backend ↔ ImageKit (*Storing media*)
- Backend → Frontend (*Send the processed information back to the user*)

1.3 Section C: Data Design

1.3.1 Object Relation Data Structure Description

Users(identity source: Clerk)

- Primary identity of the system is managed by Clerk (clerkused / token subject).
- The application does not store password
- Roles included in the system are admin, organization user, general user which are resolved using authentication token and application metadata
- Backend validates and decodes the token to determine access context

Organizations

- Stores master data/records of organizations.
- Used to group forms and manage organization-level workflows
- Fields are (id, name, description, status, createdBy, createdAt, updatedAt)
- Allow multi-tenant structure.

Tags (Field Definitions)

- Defines reusable metadata for form and profiler fields.
- Fields include (_id, key, label, type, options, rules, isRequired)
- Enables reusable field definitions, consistent validation, flexible schema evolution.

Reusable Components

- Stores shared form components created by administrators.
- Stored centrally in MongoDB (not browser-local)
- Fields are (_id, name, tagkey, type, options, rules, createdBy, createdAt, updatedAt)
- Promotes standardization. Reduce duplication and faster form building.

Forms

- Stores defined form created by form builder and is linked to specific organization

- Typical fields are (_id, organizationId, title, description, components[], label, key, type, options, rules, required)
- Also supports fully dynamics as well as configurable forms

Submissions

- Stores response submitted for forms and is linked to both form and organization
- Fields included are (_id, formId, organizationId, submittedBy, values, status, createdAt, updatedAt, values)
- Enables flexible storage of structured and semi-structured data.

User Profile

- Stores user profile data for autofill functionality with one profile per authenticated user.
- Fields includes (_id, userId, profiledata, createdAt, updatedAt)
- Includes user-defined dynamic fields.

Data relationship

Organization → Forms	1:N
Form → Submission	1:N
User → UserProfile	1:1
User → Submission	1:N
ReusableComponents → Forms	1:N

1.4 Section D: API/Interface Specifications

All the API endpoints in the Omniform application are presented in the image below. The image consists of a table which lists HTTP method (GET, POST, PUT, DELETE), description and description. The APIs are grouped into methods for admin, uploads, org, and user category.

METHOD	PATH	DESCRIPTION
ADMIN		
GET	/api/admin/whoami	Return authenticated admin identity/role context.
GET	/api/admin/orgs	List organizations for admin management.
POST	/api/admin/orgs	Create a new organization.
PUT	/api/admin/orgs/:orgId	Update organization details.
DEL	/api/admin/orgs/:orgId	Delete an organization.
GET	/api/admin/tags	List reusable field/tag definitions.
POST	/api/admin/tags	Create a new tag/field definition.
PUT	/api/admin/tags/:tagId	Update tag properties (label/type/options/rules).
DEL	/api/admin/tags/:tagId	Remove a tag definition.
GET	/api/admin/reusable-components	List saved reusable form components.
POST	/api/admin/reusable-components	Save a reusable component for all admins.
DEL	/api/admin/reusable-components/:componentId	Delete a saved reusable component.
UPLOADS		
POST	/api/uploads/profile-documents	Upload profile-related files/images.
POST	/api/uploads/form-documents	Upload form-submission files/images.
DEL	/api/uploads/imagekit-file	Delete a file from ImageKit storage.
ORG		
GET	/api/org/forms	List forms for organization-side management/review.
GET	/api/org/forms/:formId	Get a specific form definition/details.
POST	/api/org/forms/:formId/submit	Submit a filled form response.
GET	/api/org/forms/:formId/submissions	List submissions for a form.
USER		
GET	/api/user/profile	Fetch current user profile data.
PUT	/api/user/profile	Create/update current user profile data.
GET	/api/user/orgs	List organizations visible to the user.
GET	/api/user/orgs/:orgId/forms	List forms available in a selected organization.
GET	/api/user/submissions	List current user's submitted forms.

Figure 2: API Endpoints Description

1.5 Section E: Setup and Deployment

1.5.1 Local Setup Commands

Prerequisites to be installed,

Node.js 20+

Node package manager (npm)

Mongodb Atlas uri

Clerk keys

Imagekit keys

Setting up the Backend:

cd src/omniform-backend

npm install

npm run dev

Setting up the Frontend:

cd src/omniform-frontend

npm install

npm run dev

Check Build:

cd src/omniform-frontend

npm run build

1.5.2 Deployment Process

Frontend (Vercel)

1. First, we imported GitHub repository in Vercel
2. Then we set root directory to omniform-frontend
3. Set build command to “npm run build”
4. Add environment variables for frontend:

VITE_API_URL=

VITE_CLERK_PUBLISHABLE_KEY=

5. Finally, deployment was done.

Backend (Render)

1. First, we imported GitHub repository in Render
2. Then we set root directory to omniform-backend
3. Set build command to “npm install”
4. Set start command to “node server.js”
5. Add environment variables for backend:

MONGO_URI=

CLERK_PUBLISHABLE_KEY=

CLERK_SECRET_KEY=

ITE_CLERK_PUBLISHABLE_KEY=

VITE_API_URL=

FRONTEND_URL=

CORS_ORIGINS=

IMAGEKIT_PUBLIC_KEY=

IMAGEKIT_PRIVATE_KEY=

IMAGEKIT_URL_ENDPOINT=

6. Finally, deployment was done.

2 Business Documentation

2.1 Section F: Business Model Canvas

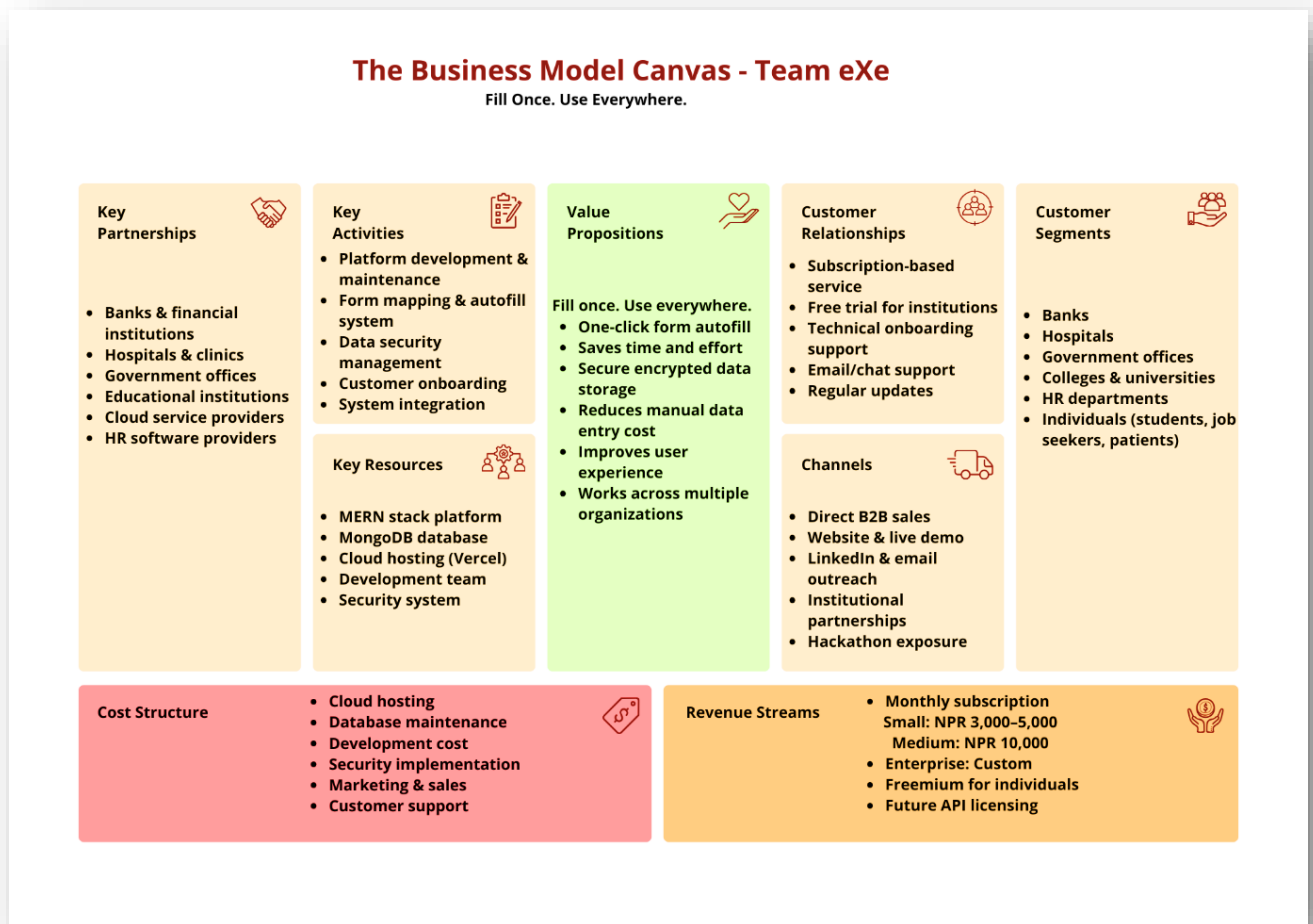


Figure 3: BMC canvas

2.2 Section G: Unique Selling Proposition

eXe provides a universal, secure identity autofill system that works across multiple organizations. It eliminates repetitive form-filling and reduces operational workload for institutions while saving time for users.

- One-click form autofill.
- Works across multiple institutions.
- Secure encrypted identity storage.

Competitor Comparison Table

Feature	eXe	Browser Autofill	Institution Forms
Works across organizations	✓	X	X
Secure centralized identity	✓	Limited	X
One-click full form fill	✓	Limited	X
Institution integration	✓	X	✓

“eXe is not just autofill — it is a universal identity layer connecting users and services.”

2.3 Section H: Market Analysis

Target Market:

eXe is designed for everyone who faces repetitive form-filling, making it a universally relevant solution.

- **Individuals:**
 - Bank customers opening accounts or applying for loans.
 - Patients at hospitals or diagnostic centers.
 - Job seekers submitting multiple applications daily.
 - Students and international applicants filling university forms.
- **Organizations:**
 - Banks, hospitals, universities, governments offices managing thousands of forms daily.
 - Early adopters: tech-savvy institutions seeking **efficiency, error reduction, and improved customer experience.**

Market Size:

- **Local TAM: 500,000+** individuals in Nepal regularly fill forms
- **Regional/ National Expansion: millions** of potential users across South Asia
- **Global opportunity: billions** of forms submitted annually, repetitive form-filling is a universal pain point.

eXe Advantages

- ✓ Works across multiple organizations (browser autofill X, institution forms X)
- ✓ Secure centralized identity storage (limited in browsers, X in institution forms)
- ✓ One-click full form fill
- ✓ Integrates seamlessly with institutions

Market Opportunity:

- Digitalization Boom: banking, healthcare, education, and government services are moving online.
- User Demand: individuals want speed and convenience, institutions want efficiency and reduced errors
- Data Privacy Awareness: secure, user-controlled identify storage is increasingly important
- Unique Hook: eXe is not just autofill, it is a universal identity layer connecting users and organizations seamlessly.

Before vs After eXe

Before eXe	After eXe
Manually filling multiple forms	One-click autofill across platforms
Errors and forgotten data	Accurate and secure data stored centrally
Frustrated users	Happy, satisfied users
Time wasted for institutions	Streamlined operations and reduced workload

Go-to-Market Strategy (First 100 Users):

- **Individuals:** social media campaigns, university collaborations, influencer endorsements.
- **Organizations:** pilot programs with banks and hospitals showcasing **measurable time savings**.
- **Viral Strategy:** encourage early adopters to share “time saved stories” with badges or recognition.
- **Messaging Focus:** emphasize **time savings, convenience, security, and efficiency**, metrics judges can instantly grasp.

2.4 Section I: Sustainability & Future Scope

1. Financial Sustainability

eXe will sustain itself through a subscription-based model for institutions that process large volumes of forms, such as banks, hospitals, universities, and government offices.

- **Revenue Model:** eXe will operate on a **subscription-based model** for organizations (banks, hospitals, universities, government offices) who process large volumes of forms.
- **Individual Users:** Free basic plan, premium features (like multi-platform integration or verified identity tags) generate additional revenue.
- **Cost Efficiency:** Cloud-based deployment (e.g., Vercel) keeps infrastructure costs low; the MERN stack minimizes development and maintenance overhead.
- **Recurring Revenue:** Institutions pay monthly/annual subscriptions, ensuring **steady, predictable income**.

2. Next Features / Product Roadmap

- **Smart Form Suggestions:** AI-powered recommendations to auto-select common responses based on user behavior.
- **Verified Identity Tag:** Optional verification for sensitive forms (banking, visa, university applications).
- **Cross-Platform Mobile App:** Android and iOS app for offline form autofill.
- **Integration API:** Allow third-party services (e.g., fintech apps) to use eXe for secure autofill.

3. Vision: 6- Month & 1- Year

6-Month Vision	1-Year Vision
Pilot with 2–3 local banks and hospitals	Expand to regional and international institutions
Achieve 5000+ active users	Scale to 50,000+ users
Launch web-based autofill for finance, healthcare, and education	Introduce mobile apps and AI-powered smart features
Collect feedback from early adopters to refine UX	Generate sustainable recurring revenue and secure partnerships

4. Partnerships & Resources for Scaling

- Banks & Hospitals: Early adopters for pilot programs
- Government Agencies: Integration with official forms and compliance
- Cloud Providers (AWS, Vercel): Scalable and secure infrastructure
- Universities / Students: Beta testers for user experience and feedback
- Data Security Firms: Advanced encryption and compliance for sensitive data